



8. Backpropagation 9. Regularization

안재국 / 2026.03.12



Computational Data Science LAB



CONTENTS

1. Backpropagation
 2. Regularization
- 
- 

01 | Backpropagation

Why do we need backpropagation?

- Gradient 계산의 필요성

- ✓ 신경망을 학습하려면 모든 가중치에 대해 오차함수의 미분값이 필요
- ✓ 하지만 오차함수에는 가중치에 직접 나타나지 않아 체인룰이 필요
- ✓ 실제 신경망은 동일한 미분 계산이 여러 번 반복됨
- ✓ 역전파는 이러한 반복 계산을 재사용하여 효율적으로 gradient를 구하는 방법

- Single-layer Networks

- ✓ 출력은 입력의 가중합으로 표현

$$y_k = \sum_i w_{ki} x_i$$

- ✓ 한 데이터에 대한 제곱오차 함수는

$$E_n = \frac{1}{2} \sum_k (y_{nk} - t_{nk})^2$$

- ✓ 특정 가중치 w_{ji} 에 대한 미분은

$$\frac{\partial y_j}{\partial w_{ji}} = x_i \quad \frac{\partial E}{\partial y_j} = y_j - t_j \quad \frac{\partial E_n}{\partial w_{ji}} = (y_{nj} - t_{nj}) x_{ni}$$

- ✓ 계산한 gradient로 가중치 업데이트

$$w_{ji} \leftarrow w_{ji} - \eta \frac{\partial E}{\partial w_{ji}}$$

01 | Backpropagation Mechanism

- multi-layer

✓ 1단계 : 순전파(feedforward) 가정: 은닉층의 활성화함수, 출력층 활성화함수 시그모이드

$$z_1 = x_1 w_1 + x_2 w_3 = 0.5 \times 0.7 + 0.3 \times 0.4 = \mathbf{0.47} \quad z_2 = x_1 w_2 + x_2 w_4 = 0.5 \times 0.3 + 0.3 \times 0.6 = \mathbf{0.33}$$

$$h_1 = \text{sigmoid}(z_1) = 0.615$$

$$z_3 = h_1 w_5 + h_2 w_6 = 0.615 \times 0.55 + 0.582 \times 0.45 = \mathbf{0.6}$$

$$h_2 = \text{sigmoid}(z_2) = 0.582$$

✓ 2단계 : 손실계산, 실제값은 1이라고 가정

$$C = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2 \quad (1 - 0.645)^2 \quad C = \mathbf{0.126}$$

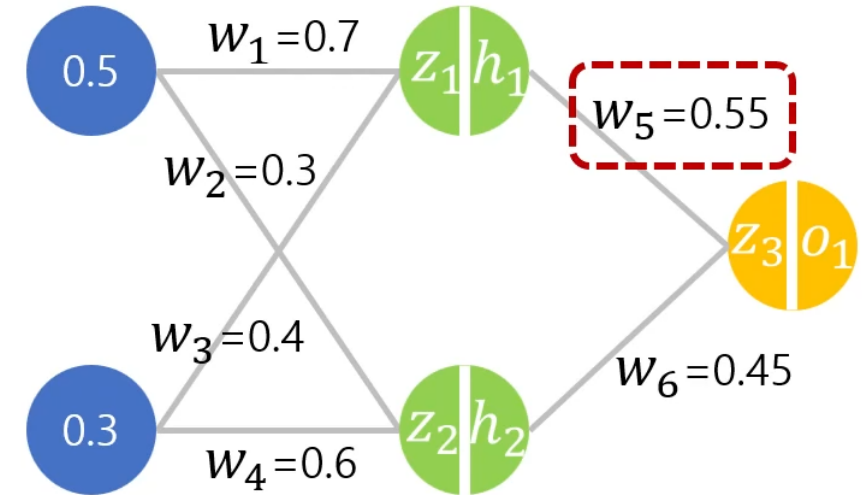
✓ 3단계 : 역전파(backpropagation)

Chain Rule을 이용해 출력에서 입력 방향으로 기울기를 전파하는 과정

✓ CHAIN RULE: 두 변수의 미분값을 구하려할때 그 관계를 모른다면

각각 아는 미분값들로 연쇄적으로 확장

$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$



01 | Backpropagation

Mechanism

$$\checkmark \quad \frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

$$\checkmark \quad \frac{\partial C}{\partial o_1} \quad C = (y - o_1)^2$$

$$\frac{\partial C}{\partial o_1} = -2(y - o_1)^{2-1}$$

$$\frac{\partial C}{\partial o_1} = -2(1 - 0.645)$$

$$= -0.71$$

$$\checkmark \quad \frac{\partial z_3}{\partial w_5} \quad z_3 = h_1 w_5 + h_2 w_6$$

$$\frac{\partial z_3}{\partial w_5} = h_1 \cancel{w_5} + \cancel{h_2 w_6}$$

$$= h_1 = \mathbf{0.615}$$

$$\checkmark \quad \frac{\partial o_1}{\partial z_3} \quad \frac{\partial O}{\partial z} = \frac{\partial}{\partial z} \left(\frac{1}{1 + e^{-z}} \right)$$

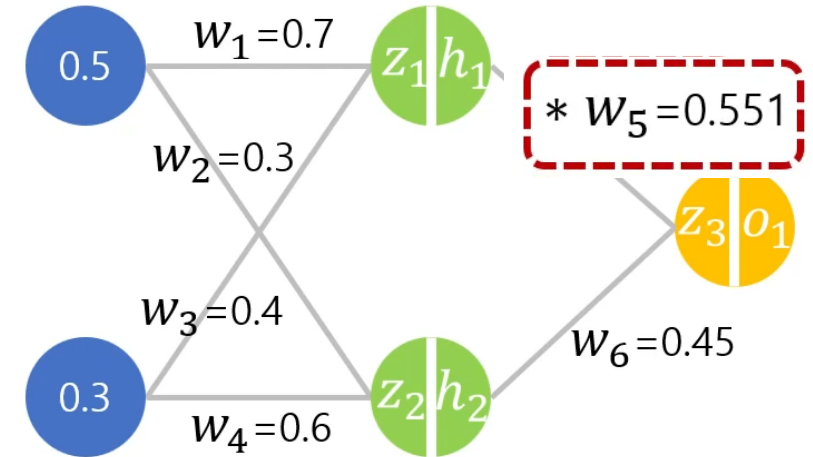
$$O(z)(1 - O(z))$$

$$0.645(1 - 0.645) = \mathbf{0.229}$$

$$\frac{\partial C}{\partial w_5} = -0.71 \cdot 0.229 \cdot 0.615 = -0.01$$

$$w_{ji} \leftarrow w_{ji} - \eta \frac{\partial E}{\partial w_{ji}}$$

$$W_5 = 0.55 + 0.01 \cdot 0.1 = 0.551$$



01 | Backpropagation Mechanism

$$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

$$\frac{\partial z_3}{\partial h_1} \quad z_3 = h_1 w_5 + h_2 w_6 \quad \cancel{h_1 w_5} + \cancel{h_2 w_6}$$

$w_5 = 0.55$

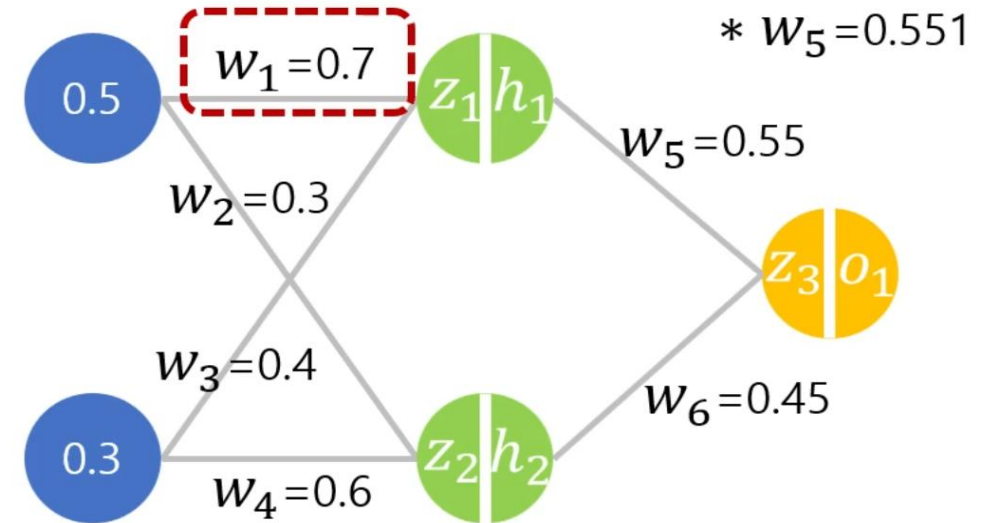
$$\frac{\partial h_1}{\partial z_1} \quad h_1(1 - h_1)$$

$$= 0.615(1 - 0.615)$$

$$= 0.237$$

$$\frac{\partial z_1}{\partial w_1} \quad z_1 = x_1 w_1 + x_2 w_3 \quad \cancel{x_1 w_1} + \cancel{x_2 w_3}$$

$x_1 = 0.5$



$$\frac{\partial C}{\partial w_1} = -0.71 \times 0.229 \times 0.55 \times 0.237 \times 0.5 = -0.0106$$

$$w_1 = 0.7 + 0.0106 \times 0.1 = 0.701$$

01 | Backpropagation

Residual Connections

- Problem: Training Very Deep Networks

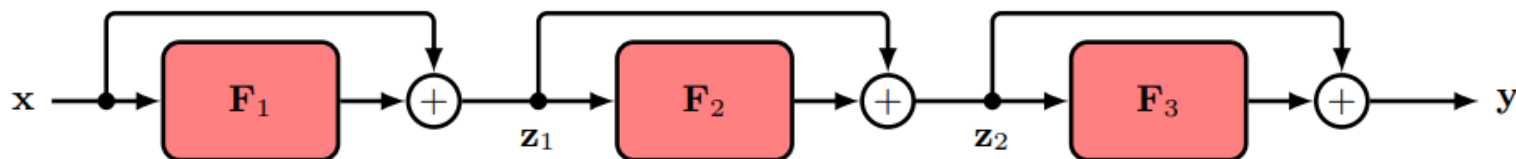
- ✓ 네트워크 깊이가 증가하면 **gradient 문제가 발생**
- ✓ 역전파 과정에서 gradient가 매우 작아짐 (vanishing)
→ 가중치 업데이트 어려움

- Key Insight

- ✓ Residual network는 출력 자체가 아니라 입력과의 차이(residual)를 학습
- ✓ gradient 전달 안정화, 매우 깊은 네트워크 학습 가능

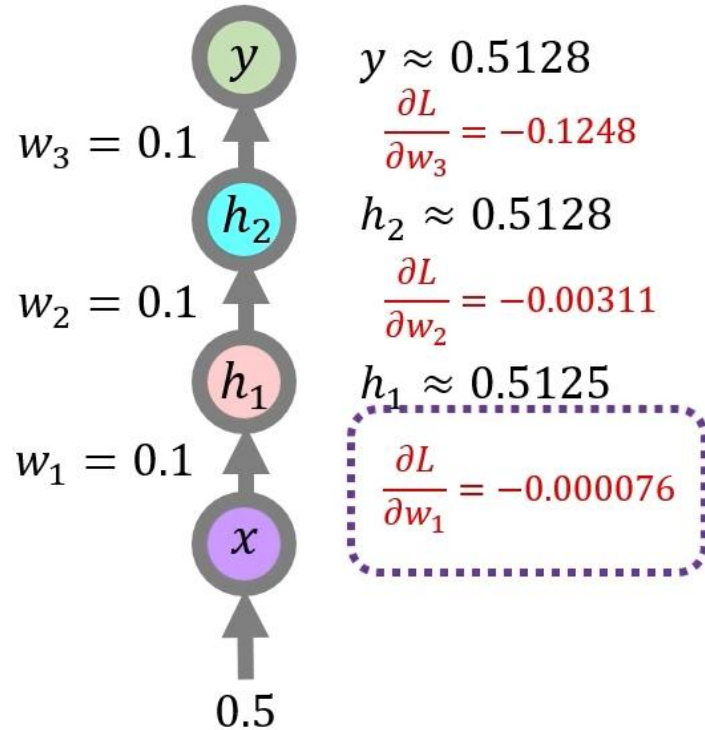
- Idea: Residual Block

- ✓ Residual connection은 입력을 그대로 더하는 **skip connection** 구조
- ✓ $y = F(x) + x$
- ✓ F : 학습할 변환 (여러 연산을 묶은 블록)

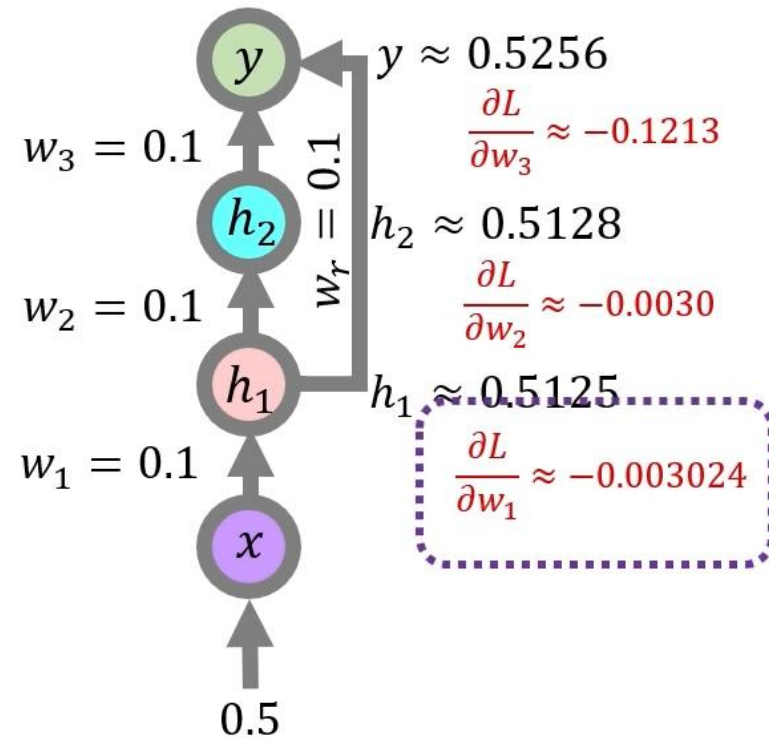


01 | Backpropagation

Residual Connections



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \left(\frac{\partial y}{\partial h_2} \frac{\partial h_2}{\partial h_1} + \frac{\partial y}{\partial h_1} \right) \frac{\partial h_1}{\partial w_1}$$

기존 경로 skip 경로

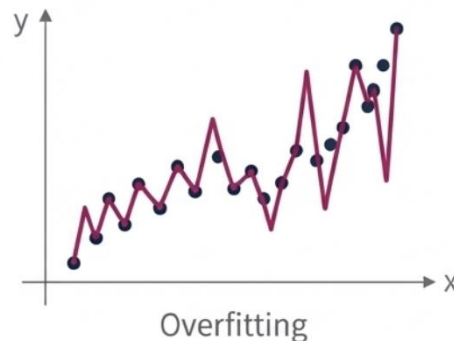
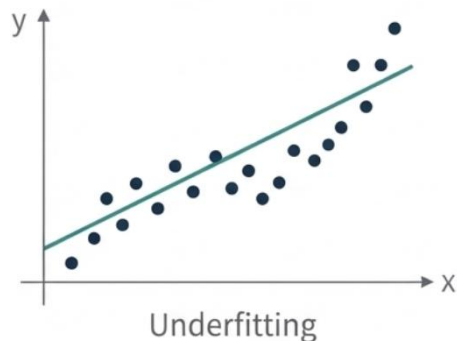


02 | Regularization

Why Do We Need Regularization?

- Underfitting vs Overfitting

- ✓ 복잡한 모델은 training data에는 매우 잘 맞음
- ✓ 하지만 unseen data에서는 성능이 크게 떨어질 수 있음
- ✓ 이는 모델이 데이터의 패턴이 아닌 noise까지 학습하기 때문



- Why Regularization?

- ✓ 모델의 복잡도를 제한하여 overfitting을 방지
- ✓ 결과적으로 generalization 성능 향상

- Main Approaches

- ✓ Weight Decay (L2 Regularization)
- ✓ Early Stopping
- ✓ Dropout / Model Averaging

02 | Regularization

Weight Decay

- Idea

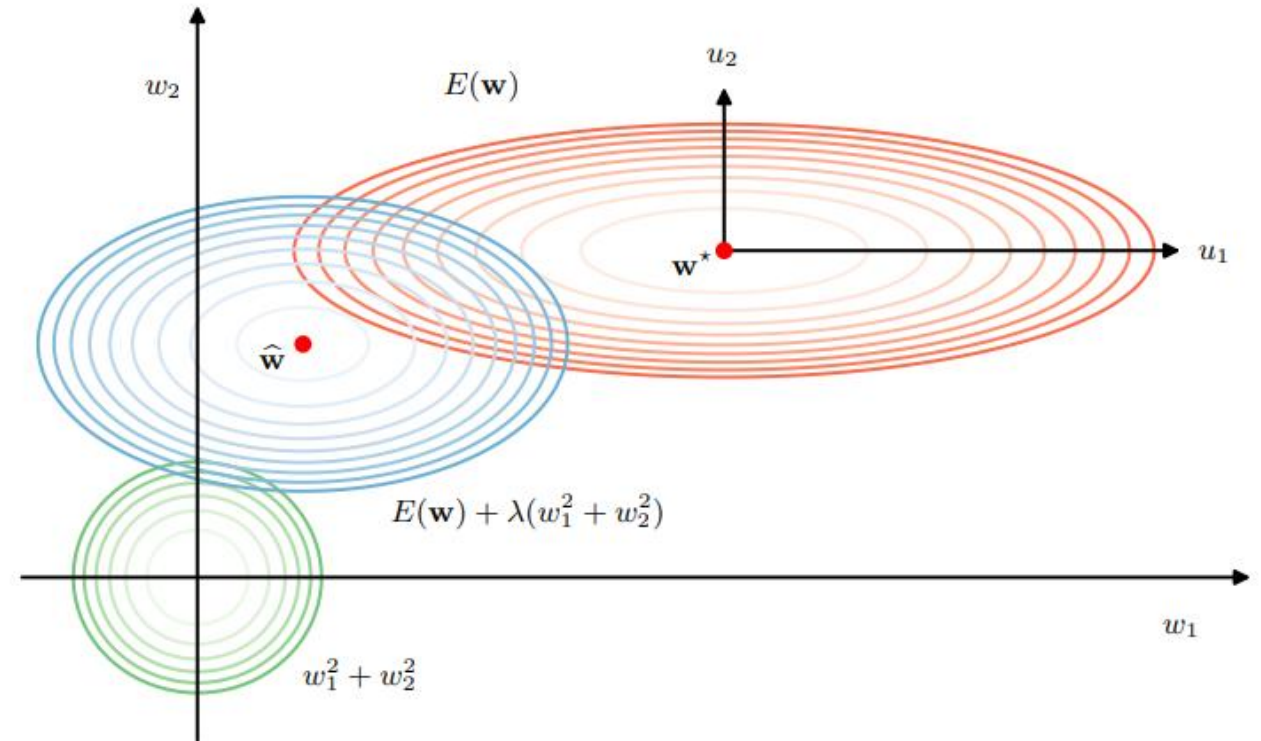
- ✓ 가중치가 지나치게 커지면 overfitting 발생
- ✓ 이를 방지하기 위해 큰 가중치에 패널티를 부여

- Regularized Error Function

- ✓ $\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{2} \sum w_i^2$
- ✓ λ : 정규화 강도 조절 파라미터

- Effect

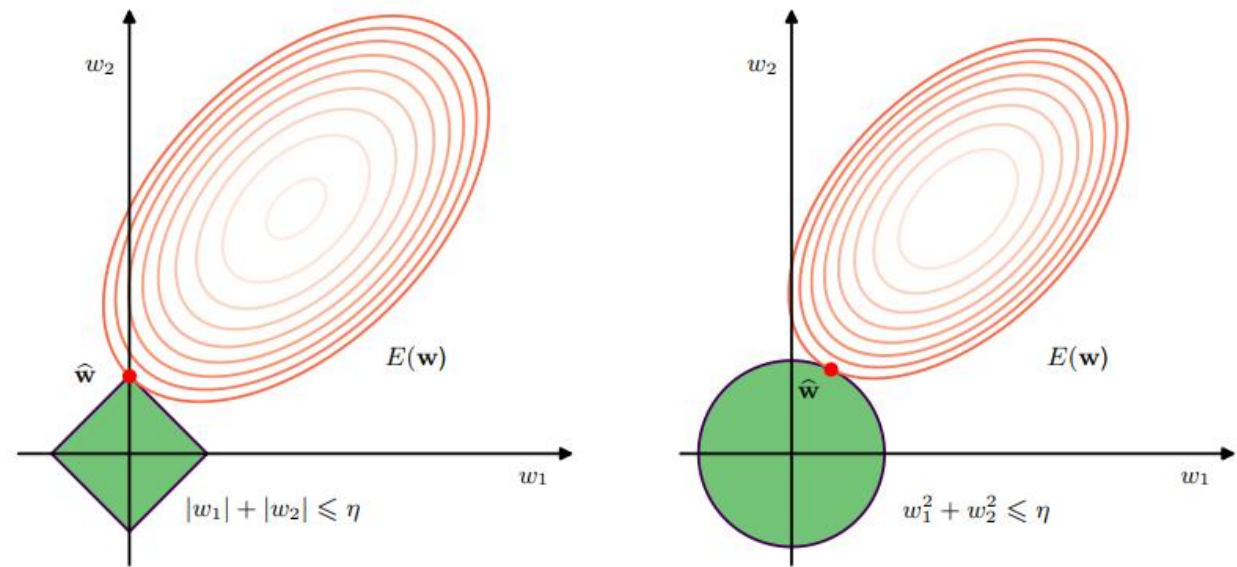
- ✓ 학습 과정에서 가중치가 0 방향으로 감소
- ✓ 중요하지 않은 파라미터의 영향 감소
- ✓ 모델 복잡도 감소 $\rightarrow$ generalization 향상



02 | Regularization

L1 vs L2 Regularization

- Idea
 - ✓ 가중치에 패널티를 부여하는 방식에 따라 L1 / L2로 구분
- Regularization Form
 - ✓ $\Omega(\mathbf{w}) = \frac{\lambda}{2} \sum |w_j|^q$
 - ✓ $q = 1 \rightarrow$ L1 Regularization (Lasso)
 - ✓ $q = 2 \rightarrow$ L2 Regularization (Weight Decay)
- Effect
 - ✓ L1은 일부 가중치를 0으로 만들며, 중요한 특성만 남김
 - ✓ L2는 가중치를 전체적으로 작게 감소시키며 모델을 부드럽게 단순화하여 안정적으로 학습



02 | Regularization

Learning Curves

- Idea

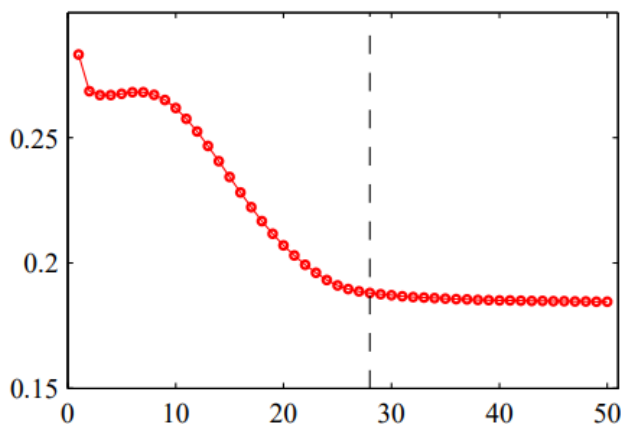
- ✓ 학습 과정에서 training error는 지속적으로 감소
- ✓ 하지만 validation error는 일정 시점 이후 증가
- ✓ 이는 모델이 training data에 과적합(overfitting) 되기 때문

- Learning Curve Interpretation

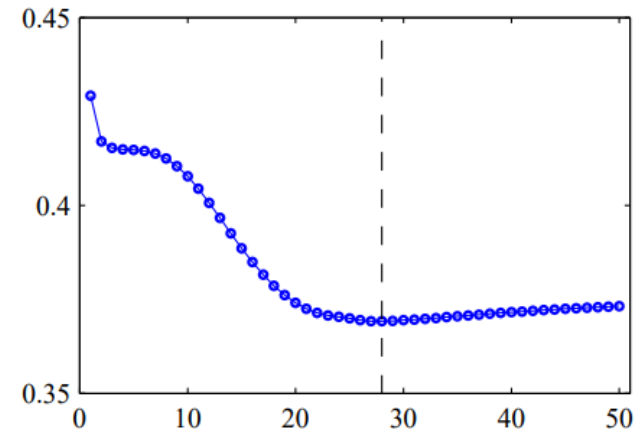
- ✓ 학습 초반 → 모델이 패턴을 학습하며
train / validation error 모두 감소
- ✓ 학습 후반 → training error는 계속 감소
→ validation error는 증가

- Key Insight

- ✓ 최적 모델은 validation error가 최소인 지점
- ✓ 이후 학습은 generalization 성능을 악화



Training error



validation error

02 | Regularization

Early Stopping

- Idea
 - ✓ validation error가 증가하기 전에 학습을 조기 종료
 - ✓ overfitting을 방지하여 generalization 성능 향상
- Key Insight
 - ✓ 학습을 오래 할수록 → model complexity 증가
 - ✓ 적절한 시점에서 멈추면 → 모델 복잡도 제어 가능

```
best_val_loss = float('inf')
counter = 0
patience = 5

for epoch in range(num_epochs):
    train_loss = train_model(model, train_loader, optimizer, criterion)
    val_loss = evaluate_model(model, val_loader, criterion)

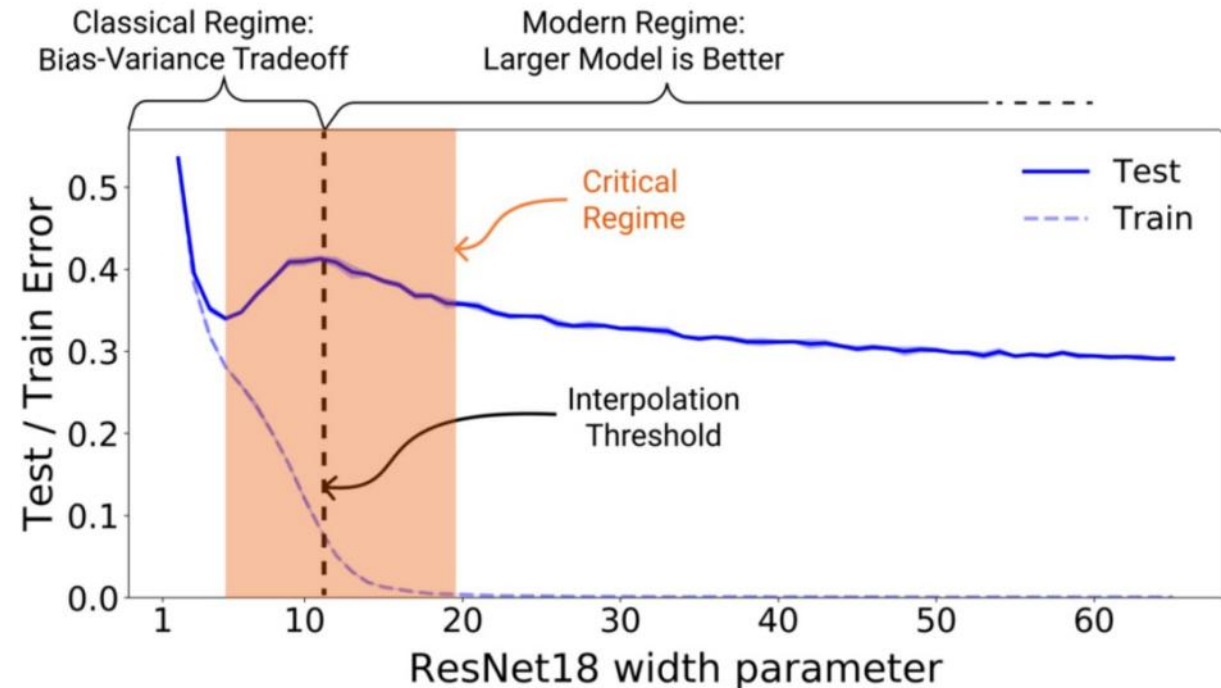
    if val_loss < best_val_loss:          # validation 성능 개선
        best_val_loss = val_loss
        counter = 0
        torch.save(model.state_dict(), "best_model.pt")
    else:                                # 개선 없음
        counter += 1

    if counter > patience:                # patience 초과 시 조기 종료
        print("Early Stopping")
        break
```

02 | Regularization

Double Descent Phenomenon

- Classical Regime
 - ✓ 모델 복잡도 증가 → test error 감소하지만 일정 수준 이후 → **overfitting 발생** 따라서 test error 다시 증가
- Critical Regime & Interpolation Threshold
 - ✓ 모델이 훈련 데이터를 거의 완벽히 맞출 수 있는 경계
 - ✓ 이 근처에서 test error가 가장 커질 수 있음
- Modern Regime
 - ✓ 충분히 큰 모델에서는 → test error가 다시 감소할 수 있음
- Regularization Strength
 - ✓ $\text{Loss} = \text{Data Loss} + \lambda \cdot \text{Regularization}$
 - ✓ 정규화 항의 가중치 λ 가 클수록 → 가중치가 더 강하게 제한됨
 - ✓ 따라서 $\lambda \uparrow = \text{모델 복잡도} \downarrow$
 - ✓ 즉, 모델 복잡도는 파라미터 수뿐 아니라 정규화 강도로도 조절될 수 있음

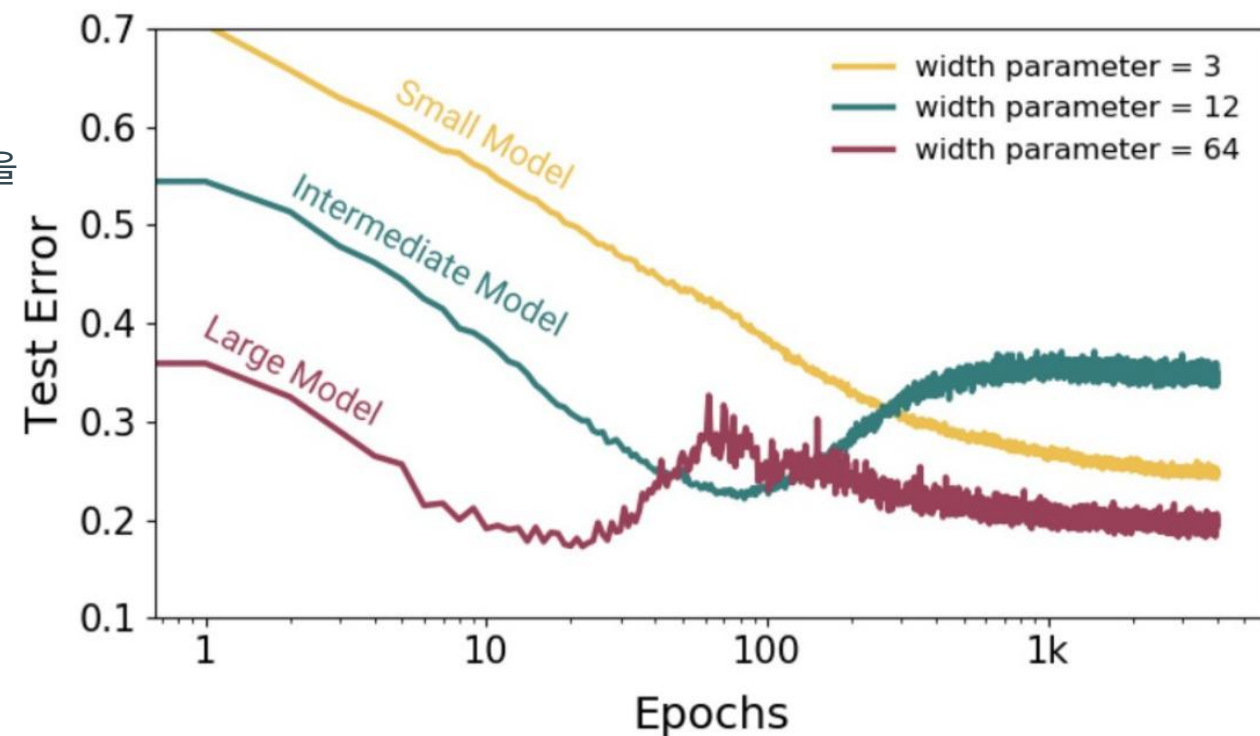


02 | Regularization

Double Descent Phenomenon

- Implication for Early Stopping

- ✓ early stopping은 유용하지만, 항상 최적의 stopping point를 보장하지는 않음
- ✓ 현재 모델이 어느 구간(Regime)에 있는지 파악하고, 최악의 성능을 내는 위험 구간을 완전히 뛰어넘도록 모델 크기, 학습 시간, 정규화 강도 밸런스를 전략적으로 설계해야 함



02 | Regularization

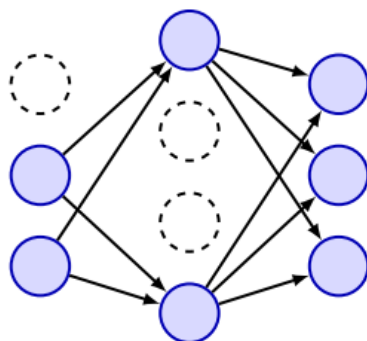
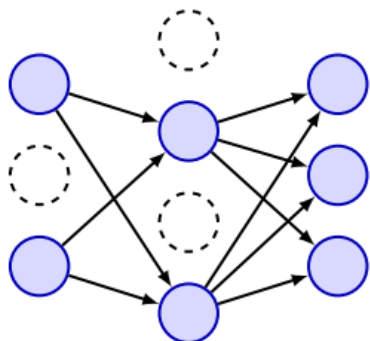
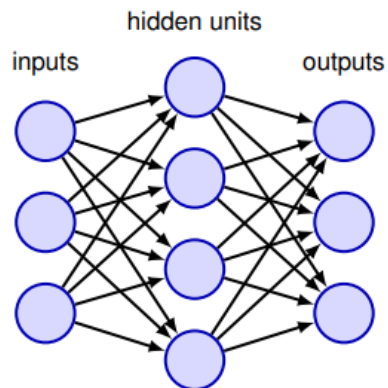
Dropout

- Idea

- ✓ 학습 과정에서 일부 노드를 랜덤하게 제거
- ✓ 노드 출력 = 0으로 설정
- ✓ 매 학습 step마다 다른 네트워크 구조 사용

- Mechanism

- ✓ 마스크 벡터 $R_i \in \{0,1\}$
- ✓ 노드는 확률 ρ 로 유지됨
- ✓ 보통 hidden layer: $\rho \approx 0.5$ input layer: $\rho \approx 0.8$



- Test Phase

- ✓ 노드를 제거하지 않고 모든 노드를 사용
- ✓ $w_{test} = \rho w \rightarrow$ 학습 시 출력과 동일하게 유지
- ✓ 출력 크기를 맞추기 위해 가중치를 ρ 로 스케일링

- Key Insight

- ✓ 2^m 개의 서로 다른 네트워크 학습 효과
- ✓ 여러 모델을 평균내는 **Model Averaging** 효과
- ✓ co-adaptation 감소 $\rightarrow$ overfitting 완화

Q&A

감사합니다.